

Техническое задание: Проектирование и реализация автономной системы защиты от биометрического спуфинга (Face Anti-Spoofing) на базе архитектуры MiniFASNetV2-SE в среде C#.NET 10 (WinForms)

Введение в проблематику уязвимостей биометрической аутентификации

В современной индустрии разработки систем контроля и управления доступом, а также программных комплексов для автоматизированного учета рабочего времени, биометрическая аутентификация по лицу стала стандартом де-факто. Однако внедрение базовых алгоритмов распознавания лиц (Face Recognition) сопряжено с фундаментальными рисками безопасности, обусловленными уязвимостью к презентационным атакам (Presentation Attacks). Традиционные нейросетевые экстракторы признаков, построенные на классических архитектурах компьютерного зрения, таких как MobileNetV4 или ResNet, оптимизированы исключительно для извлечения лицевых ориентиров (Landmarks) и вычисления векторных представлений (Embeddings) для последующего сравнения.¹ Данные модели математически не способны отличить физическое лицо живого человека от его цифровой или бумажной копии, поскольку их функция потерь при обучении не была направлена на анализ текстурной "живости" (Liveness) объекта.²

Злоумышленник, обладающий фотографией высокого разрешения или видеозаписью целевого пользователя, может с легкостью обойти систему аутентификации, продемонстрировав камере распечатанное изображение (Print Attack) или экран мобильного устройства (Replay Attack).³ Для нейтрализации данных угроз архитектура разрабатываемой системы должна интегрировать специализированный барьерный слой — подсистему Face Anti-Spoofing (FAS). В рамках данного технического задания исследуется и регламентируется процесс внедрения проекта face-antispoof-онnx, разработанного командой SuriAI, в экосистему C#.NET 10 с использованием графического интерфейса Windows Forms (WinForms).

Выбор конкретной архитектуры MiniFASNetV2-SE продиктован ее выдающейся эффективностью в условиях ограниченных вычислительных ресурсов периферийных

устройств (Edge Devices). В отличие от облачных API, которые требуют постоянного интернет-соединения и передают чувствительные биометрические данные на сторонние серверы, локальная реализация гарантирует работу в режиме "Offline-First".⁴ Это полностью исключает сетевую задержку (Network Latency) и обеспечивает соответствие строгим корпоративным стандартам конфиденциальности, так как сырые кадры с веб-камеры никогда не покидают пределов физического устройства пользователя.⁴ Компактность модели, размер которой после INT8 квантования составляет всего 600 КБ, позволяет выполнять инференс на центральном процессоре (CPU) менее чем за 20 миллисекунд даже на устаревшем оборудовании, выпущенном более десятилетия назад.²

Теоретическое обоснование архитектуры MiniFASNetV2-SE

Для корректной интеграции нейросети в программный код на языке C# необходимо глубокое понимание ее внутренних механизмов и отличий от классических систем компьютерного зрения. Архитектура MiniFASNetV2-SE представляет собой сверточную нейронную сеть, с самого начала спроектированную для решения задачи бинарной классификации: разделения входных данных на классы "Реальное лицо" (Real) и "Подделка" (Spoof).¹ Проблема обнаружения подделок сводится не к семантическому анализу геометрии лица, а к поиску микроскопических артефактов, неизбежно возникающих при попытке обмана оптического сенсора. К таким артефактам относятся муаровые узоры (Moiré patterns), генерируемые интерференцией пиксельной сетки цифрового экрана и матрицы веб-камеры, тонкие искажения цветопередачи, вызванные ограниченным цветовым охватом принтеров, а также неестественное отражение света и блики от плоских поверхностей бумаги или стекла смартфона.⁶

Чтобы заставить нейронную сеть обращать внимание на эти трудноуловимые высокочастотные паттерны, авторы архитектуры применили инновационный подход на этапе обучения, интегрировав вспомогательную ветвь потерь на основе преобразования Фурье (Fourier Transform Loss).¹ В процессе тренировки эта ветвь берет карты признаков со второго слоя сети (Stage 2) и пытается предсказать спектр Фурье, который затем сравнивается с реальным Быстрым преобразованием Фурье (FFT) исходного изображения. Этот механизм математически принуждает веса сверточных фильтров настраиваться на распознавание частотных аномалий, характерных для цифровых матриц и печатных растров, кардинально отличающихся от естественного распределения частот текстуры человеческой кожи.³ Важно отметить, что данная вспомогательная ветвь необходима исключительно для вычисления градиентов при обучении; при экспорте обученной модели в производственный формат ONNX она отсекается, что обеспечивает минимальный размер итогового графа вычислений.⁶

Структурная топология сети базируется на концепции глубинных сепарабельных сверток (Depthwise Separable Convolutions), которые значительно сокращают количество

параметров и математических операций (FLOPs) по сравнению со стандартными свертками.⁶ Сеть начинается с двух стартовых слоев, которые агрессивно снижают пространственное разрешение входного тензора с 128×128 пикселей до 64×64 , одновременно увеличивая количество каналов до 32, что позволяет существенно сэкономить вычислительные мощности на ранних этапах.⁶ Дальнейшая обработка разделена на три стадии (Stage 1, 2, 3), каждая из которых состоит из серии остаточных блоков (Residual Blocks).

Критически важным элементом, давшим название архитектуре (обозначение "SE"), является внедрение блоков Squeeze-and-Excitation. Эти блоки интегрированы на границах каждой стадии и выполняют функцию адаптивного поканального внимания. Механизм их работы заключается в применении операции глобального усредняющего пулинга (Global Average Pooling) для сжатия пространственной информации каждого канала в единичный скалярный дескриптор. Затем небольшая полносвязная подсеть анализирует эти дескрипторы и генерирует весовые коэффициенты для каждого канала. В контексте задачи антиспуфинга это означает, что если определенный сверточный фильтр специализируется на поиске текстуры бумаги, система внимания может динамически усилить сигнал именно этого канала при анализе подозрительного кадра.¹ В завершающей части сети, вместо тяжелого полносвязного слоя, применяется свертка размером 5×5 , которая сжимает признаки до вектора $1 \times 1 \times 512$, после чего классификационная голова с применением нормализации батчей (BatchNorm) и слоя прореживания (Dropout с вероятностью 0.75) выдает финальные logits.⁶

Обоснование технологического стека и библиотек

Реализация пайплайна компьютерного зрения реального времени требует применения инструментов, способных обеспечить высокую производительность операций с матрицами и безопасную интеграцию с аппаратным обеспечением. Разработка ведется на платформе C#.NET 10. Десятая итерация платформы.NET предоставляет усовершенствованные механизмы JIT-компиляции, улучшенную векторизацию (аппаратные инструкции SIMD) и оптимизированный сборщик мусора (Garbage Collector), что критически важно для предотвращения микро-задержек при обработке видеопотока со скоростью 30 кадров в секунду. Выбор Windows Forms (WinForms) в качестве каркаса пользовательского интерфейса продиктован необходимостью создания легковесного, быстро реагирующего приложения с прямым доступом к дескрипторам окон (Window Handles) для эффективной отрисовки растровых изображений в компоненте PictureBox без накладных расходов, присущих более сложным подсистемам рендеринга, таким как WPF.

Для обеспечения полного цикла захвата, математической предобработки и нейросетевого инференса в проект должны быть интегрированы следующие библиотеки

через диспетчер пакетов NuGet:

Идентификатор пакета NuGet	Назначение и функциональная роль в проекте
OpenCvSharp4	Управляемая кроссплатформенная.NET-обертка над нативной библиотекой OpenCV (C++). Предоставляет классы для захвата видео (VideoCapture), представления многомерных матриц (Mat) и выполнения аффинных преобразований (изменение размера, обрезка, конвертация цветовых пространств). ⁸
OpenCvSharp4.runtime.win	Пакет, содержащий предварительно скомпилированные бинарные файлы (DLL) нативной библиотеки OpenCV для операционных систем семейства Windows. Без этого пакета управляемая обертка не сможет вызывать неуправляемый (unmanaged) код. ⁹
OpenCvSharp4.Extensions	Библиотека расширений, критически важная для интеграции с WinForms. Содержит класс BitmapConverter, позволяющий без глубокого копирования памяти преобразовывать матрицы Mat в объекты System.Drawing.Bitmap, совместимые с элементом управления PictureBox. ¹⁰
Microsoft.ML.OnnxRuntime	Официальная библиотека от Microsoft для загрузки и выполнения моделей в формате ONNX. Обеспечивает высокоуровневый C# API (классы InferenceSession, DenseTensor) для передачи данных в вычислительный граф и получения результатов. Библиотека автоматически маршрутизирует вычисления на оптимальные аппаратные

	узлы. ¹³
--	---------------------

Библиотека Microsoft.ML.OnnxRuntime является ядром логики классификации. Формат ONNX (Open Neural Network Exchange) представляет собой открытый стандарт абстрактного синтаксического дерева вычислений, что позволяет моделям, обученным в PyTorch, бесшовно исполняться в среде.NET без необходимости портирования самого фреймворка машинного обучения.¹⁵ По умолчанию библиотека использует Execution Provider для центрального процессора (CPU), что полностью удовлетворяет требованиям архитектуры MiniFASNet, способной работать в реальном времени без графического ускорителя.²

Спецификация используемых ONNX-моделей и источники загрузки

Архитектура системы антиспуфинга строится на базе двухступенчатого конвейера (Two-Stage Pipeline). Классификатор Liveness Detection не способен самостоятельно обнаружить лица на полном кадре с камеры; он ожидает на вход строго центрированный и обрезанный фрагмент изображения, содержащий только лицо и минимально необходимое фоновое пространство.⁶ Следовательно, проекту требуются две независимые нейронные сети, работающие последовательно.

Роль в конвейере	Наименование модели	Требуемый формат	Характеристики и описание архитектуры	Источник для скачивания файлов
Детекция лиц (Face Detection)	YuNet	.onnx	Сверхлегкий и высокоточный детектор лиц. Извлекает координаты ограничивающего прямоугольника (Bounding Box) и пяти ключевых точек лица (Landmarks). Оптимизиров	Репозиторий OpenCV Zoo (файл face_detection_yunet_2023mar.onnx). ¹⁷

			ан для работы через модуль CvDnn в OpenCV. ¹⁷	
Антиспуфинг (Liveness Detection)	MiniFASNetV2-SE	.onnx	Основной классификатор "Живое/Подделка". Анализирует текстуру входного обрезанного изображения. Размер файла около 1.8 МБ (или 600 КБ для INT8). Ожидает на вход тензор с коэффициентом масштабирования обрезки 2.7. ⁶	Страница релизов проекта(https://github.com/yakhyo/face-anti-spoofing/tree/main/weights) или оригинального (https://github.com/SuriAI/face-antispoof-onnx). ²⁰

Оригинальная модель MiniFASNetV2-SE была обучена в PyTorch и сохранена с весами .pth, однако для использования в C#.NET требуются предварительно экспортированные файлы .onnx. Если в репозиториях представлены только файлы PyTorch, разработчику потребуется однократно выполнить скрипт экспорта onnx_export.py, предоставляемый в проекте, чтобы сгенерировать ONNX-представление графа с фиксированными или динамическими осями пакетов (batch axes).²⁰ Использование именно модели V2-SE с коэффициентом обрезки (Crop Scale) 2.7 доказало свою наибольшую устойчивость в реальных условиях.²⁰

Пошаговые этапы реализации программного комплекса

Процесс разработки разделен на строгие последовательные этапы, гарантирующие правильную инициализацию аппаратных ресурсов, корректную математическую обработку тензоров и синхронизацию многопоточных операций с графическим

интерфейсом.

Этап 1: Проектирование архитектуры пользовательского интерфейса WinForms

Создание базового каркаса начинается с инициализации нового проекта типа "Windows Forms App" с целевой платформой .NET 10. В главном классе формы (Form1) необходимо разместить ключевые элементы управления. Центральным визуальным компонентом является PictureBox (например, pictureBoxCamera), свойство SizeMode которого должно быть установлено в Zoom, чтобы сохранять пропорции видеопотока при изменении размера окна. Ниже размещается панель управления с кнопками инициализации (btnStartCamera) и остановки захвата (btnStopCamera). Для отображения телеметрии и результатов инференса добавляются элементы Label: lblStatus (вывод вердикта "Реальное лицо" или "Атака"), lblConfidence (уверенность нейросети в процентах) и lblFPS (количество обработанных кадров в секунду).

Критическим архитектурным решением на данном этапе является проектирование потоковой модели. Захват кадров с веб-камеры через OpenCV и выполнение вычислений в ONNX Runtime являются синхронными блокирующими операциями (CPU-bound). Если попытаться разместить этот цикл в обработчике события кнопки на главном потоке пользовательского интерфейса (UI Thread), приложение мгновенно перестанет отвечать на запросы операционной системы ("зависнет").¹⁰ Вся логика конвейера должна быть инкапсулирована в отдельный фоновый поток, создаваемый с помощью класса Thread или современной абстракции Task.Run. Поскольку фоновый поток не имеет права напрямую изменять свойства элементов управления WinForms (например, присваивать новый кадр в PictureBox.Image или изменять текст в Label), необходимо использовать механизм маршалинга вызовов. Это реализуется через проверку свойства InvokeRequired и последующий вызов метода Invoke с передачей делегата MethodInvoker, что безопасно перенаправляет обновление интерфейса обратно в контекст главного потока.¹²

Этап 2: Инициализация видеозахвата и управление жизненным циклом матриц

Для получения видеопотока применяется класс VideoCapture из пространства имен OpenCvSharp. Экземпляр этого класса инициализируется с параметром 0, что дает команду операционной системе Windows открыть доступ к веб-камере по умолчанию.¹¹ Если требуется поддержка нескольких камер, индекс может быть динамически изменен. Метод Open() активирует аппаратный датчик.

Внутри фонового потока запускается бесконечный цикл while (isCameraRunning). На каждой итерации цикла вызывается метод _capture.Read(_image), который считывает текущий кадр в объект Mat.¹⁰ Здесь кроется важнейший аспект управления памятью в .NET при работе с OpenCV. Объект Mat представляет собой управляемую оболочку над

неуправляемым (unmanaged) указателем на фрагмент оперативной памяти C++. Создание нового объекта Mat на каждом кадре приведет к катастрофической утечке памяти и исчерпанию ресурсов в течение нескольких секунд, так как сборщик мусора.NET не всегда успевает вовремя вызывать финализаторы для неуправляемых ресурсов. Правильным архитектурным паттерном является предварительное создание единственного объекта `private Mat _image = new Mat();` на уровне класса формы и его многократное переиспользование (in-place modification) в методе Read.¹¹ После успешного чтения кадра производится проверка `if (_image.Empty()) continue;` для игнорирования поврежденных или пустых буферов.

Этап 3: Интеграция модуля локализации лиц YuNet

Полученный сырой кадр извлекается из объекта `_image` и передается в подсистему Face Detection. Для этого заранее, при инициализации формы, должен быть загружен граф модели YuNet с использованием подсистемы глубокого обучения OpenCV. Создается экземпляр `FaceDetectorYN`, инициализируемый путем к файлу `face_detection_yunet_2023mar.onnx`.¹⁷ При инициализации детектора необходимо передать базовые конфигурации: размер входного изображения, который должен совпадать с разрешением камеры (например, 640×480), порог уверенности (Score Threshold, оптимальное значение 0.6 для отсекация ложных срабатываний) и порог подавления немаксимумов (NMS Threshold, около 0.3 , чтобы алгоритм не обводил одно и то же лицо несколькими рамками).¹⁷

В рабочем цикле кадр передается в метод `Detect` детектора YuNet. Метод возвращает многомерную матрицу, где каждая строка содержит информацию об одном обнаруженном лице. Данные структурированы строго: первые четыре элемента строки представляют координаты ограничивающего прямоугольника (Bounding Box) в формате $X_{min}, Y_{min}, Width, Height$. Следующие десять элементов представляют координаты по осям X и Y для пяти ключевых ориентиров лица (правый глаз, левый глаз, кончик носа, правый угол рта, левый угол рта). Последний элемент строки содержит общую вероятность детекции.²

Поскольку система контроля доступа чаще всего анализирует человека, непосредственно взаимодействующего с терминалом, программная логика должна отсортировать все обнаруженные лица по площади их ограничивающего прямоугольника ($Width \times Height$) и передавать на следующий этап антиспуфинга только самую крупную рамку, игнорируя лица людей, проходящих на заднем фоне.

Этап 4: Математическая предобработка, обрезка и масштабирование (Preprocessing)

Это наиболее критически важный этап, определяющий финальную точность модели Liveness Detection. Классификатор MiniFASNetV2-SE был натренирован не на произвольных лицах, а на строго стандартизированных квадратных патчах (Crops). Если рамка лица обрезана слишком плотно (например, захвачены только глаза и нос), нейросеть потеряет важнейший периферийный контекст, такой как прямые грани мобильного телефона или рамки печатной фотографии, которые часто выдают презентационную атаку. Если же рамка вырезана слишком широко, лицо займет малую часть тензора, и после уменьшения разрешения текстурные артефакты муара будут уничтожены.⁶

Согласно спецификациям авторов архитектуры, для модели V2-SE применяется коэффициент масштабирования рамки (Crop Scale), равный **2.7**.²⁰ Программная реализация этого масштабирования требует выполнения точных геометрических вычислений. Сначала вычисляются координаты центра исходного Bounding Box: $C_x = X_{min} + Width/2$ и $C_y = Y_{min} + Height/2$. Затем определяется максимальная сторона исходного прямоугольника, чтобы гарантировать получение строгой квадратной формы: $S_{max} = \max(Width, Height)$. Новая длина стороны вычисляется путем умножения: $S_{new} = S_{max} \times 2.7$. Наконец, вычисляются новые координаты верхнего левого угла для квадратной обрезки: $X_{new} = C_x - S_{new}/2$ и $Y_{new} = C_y - S_{new}/2$.

При выполнении этих расчетов новые координаты могут выйти за границы исходного кадра (например, если лицо находится слишком близко к краю матрицы камеры). В таких ситуациях алгоритм должен применять стратегию заполнения (Padding) недостающих областей пикселями черного цвета или осуществлять безопасную обрезку по границам кадра. Полученная квадратная область извлекается из исходной матрицы с помощью конструкции индексатора Mat.

Далее вырезанный фрагмент должен быть приведен к строгому размеру, который ожидает входной слой нейросети ONNX. Для архитектуры MiniFASNetV2 этот размер обычно составляет 128×128 или 80×80 пикселей.¹ Изменение размера выполняется методом Cv2.Resize с использованием билинейной интерполяции (InterpolationFlags.Linear), которая обеспечивает оптимальный баланс между сохранением текстурной информации и скоростью выполнения.¹

Этап 5: Формирование входного тензора и выполнение графа вычислений (ONNX Inference)

Библиотеки компьютерного зрения, такие как OpenCV, считывают и хранят изображения в цветовом пространстве BGR (Blue, Green, Red) и в формате упаковки памяти HWC (Height,

Width, Channels).¹⁴ Нейронные сети, обученные в фреймворке PyTorch и экспортированные в стандарт ONNX, математически ожидают поступление данных в цветовом пространстве RGB и в плоском непрерывном тензорном формате NCHW (Number of batches, Channels, Height, Width).¹⁴ Трансформация форматов является строго обязательной, иначе сеть получит искаженные цветовые и пространственные данные, что приведет к полностью случайным предсказаниям.

Процесс подготовки данных в C# реализуется следующим образом:

1. Цветовое пространство обрезанного фрагмента лица преобразуется вызовом `Cv2.CvtColor(croppedFace, rgbFace, ColorConversionCodes.BGR2RGB)`.
2. Выделяется непрерывный одномерный массив в управляемой памяти `float tensorData = new float;`
3. Осуществляется двойной цикл итерации по пикселям матрицы. Пиксельные значения извлекаются и конвертируются из байтового формата (от 0 до 255) в формат с плавающей запятой (Float32).
4. В процессе цикла выполняется нормализация значений. Нейронные сети не оперируют большими целыми числами, им требуются значения с нулевым средним и единичной дисперсией. Для моделей MiniFASNet часто применяется нормализация ImageNet (вычитание средних значений $\text{Mean} = [0.485, 0.456, 0.406]$ и деление на стандартное отклонение $\text{StdDev} = [0.229, 0.224, 0.225]$) или простое масштабирование в диапазон $[-1, 1]$ путем деления каждого пикселя на $255.0f$.²⁴
5. Нормализованные значения записываются в одномерный массив `tensorData` с математическим смещением индексов таким образом, чтобы сформировать структуру NCHW. Сначала последовательно записываются все пиксели красного канала, затем зеленого, затем синего.¹⁴
6. На основе подготовленного массива конструируется объект `DenseTensor<float>(tensorData, new int { 1, 3, Height, Width })` из библиотеки `Microsoft.ML.OnnxRuntime`.

Выполнение вычислительного графа осуществляется через объект `InferenceSession`. Этот объект должен быть инициализирован единожды при старте приложения, так как процесс загрузки файла `.onnx` с диска и выделения памяти под веса является крайне ресурсоемкой операцией.²⁶ Метод `session.Run()` принимает на вход коллекцию именованных тензоров `NamedOnnxValue.CreateFromTensor("input", inputTensor)`, где строка "input" должна строго соответствовать имени входного узла, зафиксированному при экспорте графа.²² В процессе выполнения прямого прохода (Forward Pass) активируются все механизмы модели: глубокие свертки, блоки Squeeze-and-Excitation и финальный полносвязный слой.

Этап 6: Постобработка логитов и реализация бизнес-логики принятия решений

Результатом выполнения функции `session.Run()` является коллекция выходных тензоров. Из нее извлекается единственный тензор (обычно с именем "output"), который содержит вектор сырых математических оценок — логитов (Logits) размером 1x2. Архитектура MiniFASNetV2 завершается полносвязным слоем, который уменьшает размерность высокоуровневых признаков с 512 до 128, а затем с 128 до 2, что соответствует двум целевым классам: Real и Spoof.⁶ Порядок меток зависит от скрипта, использовавшегося для экспорта ONNX, но доминирующим стандартом в проекте является назначение индекса 0 для класса Fake/Spoof и индекса 1 для класса Real.⁶

Сырые логиты могут принимать любые значения в диапазоне от $-\infty$ до $+\infty$. Для интерпретации этих значений в виде понятных вероятностей от 0 до 1, необходимо программно реализовать функцию нормализации Softmax. Математически для вектора логитов L вероятность класса i вычисляется по формуле:

$$P_i = \frac{e^{L_i}}{\sum_{j=0}^1 e^{L_j}}$$

В коде на языке C# эта операция сводится к вычислению экспоненты `Math.Exp()` для каждого из двух логитов, нахождению их суммы и делению экспоненты логита класса Real на эту общую сумму. Результатом является значение P_{Real} , отражающее степень уверенности нейросети в том, что перед камерой находится живой человек.

Финальным шагом является реализация бизнес-логики на основе порогового значения (Thresholding). Системы безопасности биометрического контроля не могут оперировать вероятностями "пятьдесят на пятьдесят". Для классификатора MiniFASNet рекомендуется устанавливать жесткий порог уверенности (например, 0.75 или 0.80).²⁸ В программном коде формируется условный оператор: если P_{Real} превышает 0.80, система авторизует пользователя и передает управление бизнес-логике (например, открытие турникета или фиксация прихода сотрудника). Если значение ниже порога, система мгновенно блокирует доступ и выводит в графический интерфейс WinForms (в компонент `IblStatus`) предупреждение об обнаружении презентационной атаки. Дополнительно на кадре `pictureBoxCamera` цвет ограничивающего прямоугольника вокруг лица может быть изменен с зеленого на красный с помощью методов `Cv2.Rectangle` и `Cv2.PutText`.¹⁹ Финальный кадр конвертируется в `Bitmap` через `BitmapConverter.ToBitmap` и отрисовывается в интерфейсе.¹⁰

Стратегия тестирования, метрики и валидация системы

Внедрение биометрических технологий требует применения строго формализованных процедур гарантии качества (Quality Assurance), которые существенно сложнее стандартного модульного тестирования кода. Поскольку система антиспуфинга будет развернута на периферийном оборудовании в условиях неконтролируемой среды (Edge Deployment), стратегия тестирования должна охватывать оценку производительности, метрическую валидацию на наборах данных и эмпирические испытания физическими атаками.¹

1. Бенчмаркинг производительности и управление памятью: Профилирование программного обеспечения в среде.NET должно сфокусироваться на измерении сквозной задержки (End-to-End Latency). Тестирование должно подтвердить, что полный конвейер выполнения, включающий захват кадра с веб-камеры, детекцию лица алгоритмом YuNet, математическую обрезку, формирование тензора и вычисление логитов в ONNX Runtime MiniFASNetV2, занимает не более 50-60 миллисекунд. Такая производительность обеспечит стабильную отрисовку интерфейса WinForms с частотой не менее 15-20 кадров в секунду, что воспринимается пользователем как плавное видео.² Крайне важным аспектом является использование инструмента Visual Studio Diagnostic Tools для мониторинга управляемой и неуправляемой памяти. Тестировщик должен оставить приложение работающим с активной камерой на несколько часов, чтобы гарантировать отсутствие утечек памяти, связанных с забытыми вызовами методов Dispose() для объектов Mat из OpenCV.⁹

2. Метрическая валидация на стандартизированных наборах данных: Перед развертыванием в продуктивную среду модель и алгоритм предварительной обработки должны быть провалидированы на крупных академических бенчмарках. Оригинальная архитектура MiniFASNet оценивалась на наборе данных CelebA-Spoof, содержащем более 70 000 размеченных примеров, и достигла точности около 98%.² Качество системы измеряется через три фундаментальные биометрические метрики:

Метрика	Расшифровка	Влияние на безопасность системы
FAR	False Acceptance Rate (Коэффициент ложного пропуска)	Вероятность того, что фотография, силиконовая маска или экран смартфона будут ошибочно распознаны как живой человек. Это самая критичная уязвимость; значение FAR должно стремиться к

		абсолютному нулю. ³¹
FRR	False Rejection Rate (Коэффициент ложного отказа)	Вероятность того, что авторизованный живой сотрудник будет ошибочно заблокирован системой. Высокий показатель FRR не нарушает безопасность, но разрушает пользовательский опыт (UX), заставляя людей многократно пытаться пройти аутентификацию. ³¹
EER	Equal Error Rate (Равная частота ошибок)	Точка на графике ROC-кривой, в которой значения FAR и FRR пересекаются. Служит универсальным индикатором общей эффективности алгоритма. Настройка системы сводится к выбору порогового значения (Threshold), которое минимизирует FAR, удерживая FRR на приемлемом уровне.

3. Эмпирическое тестирование физическими атаками (Physical Presentation Attacks):

Академические метрики не всегда отражают реальное поведение алгоритмов компьютерного зрения "в поле". Инженеры QA должны провести ручное тестирование, симулируя действия злоумышленников, пытающихся обмануть веб-камеру.³ Сценарии должны включать:

- *Оценка Print Attack*: Предъявление камере распечатанных цветных фотографий целевого пользователя в форматах A4 и A3. Тестирование должно проводиться как с матовой, так и с глянцевой бумагой. Глянцевая бумага генерирует специфические отражения света, в то время как матовая демонстрирует плоскую текстуру целлюлозы. Дополнительно тестируется сгибание бумаги (Warped Paper Attack) для имитации трехмерного объема человеческого лица.³
- *Оценка Replay Attack (Screen Spoofing)*: Демонстрация видеозаписей лица через

экраны цифровых устройств. В тестах должны участвовать дисплеи различных типов матриц (OLED, IPS, TN) и разной плотности пикселей (PPI). Экраны генерируют высокочастотный муар и излучают собственный свет, что должно уверенно перехватываться нейросетью благодаря её фурье-тренировке.⁶

- *Стресс-тестирование оптических условий:* Оценка работы алгоритмов детекции YuNet и классификатора MiniFASNet при агрессивном контрольном освещении (Backlight), боковых тенях, а также при отклонении головы пользователя от фронтальной оси объектива более чем на 30 градусов по горизонтали и вертикали, что является документально подтвержденным ограничением архитектуры.⁷

Заключение

Интеграция комплекса Face Anti-Spoofing на платформе C#.NET 10 с использованием графической подсистемы Windows Forms представляет собой передовое архитектурное решение для обеспечения безупречной безопасности биометрической идентификации. Применение конвейера из легковесного детектора лиц YuNet и высокоспециализированной нейросети MiniFASNetV2-SE в формате ONNX обеспечивает непревзойденный баланс между математической точностью выявления презентационных атак (достигающей 98% на стандартизированных наборах данных) и экстремально низким потреблением вычислительных ресурсов.¹

Скрупулезная реализация всех описанных этапов — от управления потоками и жизненным циклом неуправляемых матриц OpenCV до строгого математического масштабирования обрезки с коэффициентом 2.7 и трансформации тензоров в формат NCHW — гарантирует устойчивость программного продукта к наиболее изощренным типам спуфинга, включая качественную полиграфию и цифровые видео-атаки высокого разрешения.³

Фундаментальное архитектурное преимущество заключается в том, что система способна выполнять весь цикл вычислений локально, непосредственно на оборудовании пользователя. Данный подход "Offline-First" не только полностью ликвидирует зависимость от скорости интернет-соединения, но и удовлетворяет самым строгим корпоративным и законодательным требованиям к конфиденциальности, исключая любую передачу сырых биометрических данных за пределы доверенного контура.⁴

Источники

1. GitHub - SuriAI/face-antispoof-onnx: Ultra-lightweight (600KB) Face Anti-Spoofing classifier. Optimized MiniFASNetV2-SE implementation validated on 70k+ samples with, дата последнего обращения: марта 12, 2026, <https://github.com/suriAI/face-antispoof-onnx>
2. I spent a month training a lightweight Face Anti-Spoofing model that runs on low end machines : r/learnmachinelearning - Reddit, дата последнего обращения: марта 12, 2026, https://www.reddit.com/r/learnmachinelearning/comments/1px88ci/i_spent_a_mo

- [nth_training_a_lightweight_face/](#)
3. Silent-Face-Anti-Spoofing/README_EN.md at master · minivision-ai ..., дата последнего обращения: марта 12, 2026,
https://github.com/minivision-ai/Silent-Face-Anti-Spoofing/blob/master/README_EN.md
 4. suriAI/suri: #1 Free and OSS AI-powered attendance tracking with real-time face recognition, anti-spoofing protection, and offline-first architecture. Designed for real-world environments. - GitHub, дата последнего обращения: марта 12, 2026, <https://github.com/suriAI/suri>
 5. Built a lightweight Face Anti Spoofing layer for my AI project : r/computervision - Reddit, дата последнего обращения: марта 12, 2026,
https://www.reddit.com/r/computervision/comments/1px7u7t/built_a_lightweight_face_anti_spoofing_layer_for/
 6. face-antispoof-onnx/docs/ARCHITECTURE.md at main - GitHub, дата последнего обращения: марта 12, 2026,
<https://github.com/suri-ph/face-antispoof-onnx/blob/main/docs/ARCHITECTURE.md>
 7. silent-face-anti-spoofing model | Clarifai - The World's AI, дата последнего обращения: марта 12, 2026,
<https://clarifai.com/minivision-ai/silent-face-anti-spoofing/models/silent-face-anti-spoofing>
 8. Face Detection Using C# and OpenCVSharp - Practical ML.NET User Group 01/19/2022, дата последнего обращения: марта 12, 2026,
<https://www.youtube.com/watch?v=THwnIz2PEvE>
 9. How to use OpenCV in .NET with OpenCVSharp - Luis Llamas, дата последнего обращения: марта 12, 2026,
<https://www.luisllamas.es/en/how-to-use-opencv-in-net-with-opencvsharp/>
 10. c# Opencvsharp 3 camcorder capture on windows form - Stack Overflow, дата последнего обращения: марта 12, 2026,
<https://stackoverflow.com/questions/39450823/c-sharp-opencvsharp-3-camcorder-capture-on-windows-form>
 11. dotnet – Display the camera feed in a WinForm using #OpenCV and #net5 - El Bruno, дата последнего обращения: марта 12, 2026,
<https://elbruno.com/2020/11/16/dotnet-display-the-%F0%9F%8E%A6-camera-feed-in-a-winform-using-opencv-and-net5/>
 12. how to display the camera feed in C# using OpenCvSharp - YouTube, дата последнего обращения: марта 12, 2026,
<https://www.youtube.com/watch?v=cOY-XvzV79o>
 13. Inference Stable Diffusion with C# and ONNX Runtime, дата последнего обращения: марта 12, 2026,
<https://onnxruntime.ai/docs/tutorials/csharp/stable-diffusion-csharp.html>
 14. C# pre and postprocessing with opencvsharp for onnxruntime #11411 - GitHub, дата последнего обращения: марта 12, 2026,
<https://github.com/microsoft/onnxruntime/discussions/11411>
 15. Developing a face recognition application with ONNX models | by Ankit Khedia |

- Apache MXNet | Medium, дата последнего обращения: марта 12, 2026,
<https://medium.com/apache-mxnet/onnx-model-zoo-developing-a-face-recognition-application-with-onnx-models-64eeeddb9c7a>
16. GitHub - onnx/models: A collection of pre-trained, state-of-the-art models in the ONNX format, дата последнего обращения: марта 12, 2026,
<https://github.com/onnx/models>
 17. What's the Best Face Detector?. Comparing Dlib, OpenCV, MTCNN, and... | by Amos Stailey-Young | Python's Gurus | Medium, дата последнего обращения: марта 12, 2026,
<https://medium.com/pythons-gurus/what-is-the-best-face-detector-ab650d8c1225>
 18. mawax/face-detection-yunet - GitHub, дата последнего обращения: марта 12, 2026, <https://github.com/mawax/face-detection-yunet>
 19. Real-time face detection with YuNet and C# - Matthijs Wagemakers, дата последнего обращения: марта 12, 2026,
<https://www.wagemakers.net/posts/real-time-face-detection-yunet-csharp/>
 20. yakhyo/face-anti-spoofing - GitHub, дата последнего обращения: марта 12, 2026, <https://github.com/yakhyo/face-anti-spoofing>
 21. Face-Recognition Training Framework, дата последнего обращения: марта 12, 2026, <https://yakhyo.github.io/face-recognition/>
 22. onnx_export.py - yakhyo/face-anti-spoofing - GitHub, дата последнего обращения: марта 12, 2026,
https://github.com/yakhyo/face-anti-spoofing/blob/main/onnx_export.py
 23. How to take snapshots with the web camera with C# using the OpenCVSharp library in WinForms | Our Code World, дата последнего обращения: марта 12, 2026,
<https://ourcodeworld.com/articles/read/761/how-to-take-snapshots-with-the-web-camera-with-c-sharp-using-the-opencvsharp-library-in-winforms>
 24. How to calculate the mean and standard deviation - Normalizing datasets in Pytorch, дата последнего обращения: марта 12, 2026,
<https://towardsdatascience.com/how-to-calculate-the-mean-and-standard-deviation-normalizing-datasets-in-pytorch-704bd7d05f4c/>
 25. [Discussion] Why normalise according to ImageNet mean and std dev for transfer learning?, дата последнего обращения: марта 12, 2026,
<https://discuss.pytorch.org/t/discussion-why-normalise-according-to-imagenet-mean-and-std-dev-for-transfer-learning/115670>
 26. Efficient Net V2 M ONNX model infers significantly slower on small input, дата последнего обращения: марта 12, 2026,
<https://datascience.stackexchange.com/questions/130511/efficient-net-v2-m-onnx-model-infers-significantly-slower-on-small-input>
 27. Face Anti Spoofing Computer Vision Model - Roboflow Universe, дата последнего обращения: марта 12, 2026,
<https://universe.roboflow.com/face-anti-spoofing/face-anti-spoofing-gnl8a>
 28. hairyamax/Face-AntiSpoofing: Face Anti-Spoofing project. Lanit-Tercom summer school 2022 - GitHub, дата последнего обращения: марта 12, 2026,

- <https://github.com/hairymax/Face-AntiSpoofing>
29. face-anti-spoofing/onnx_inference.py at main · yakhyo/face-anti ..., дата последнего обращения: марта 12, 2026,
https://github.com/yakhyo/face-anti-spoofing/blob/main/onnx_inference.py
30. Pull requests · suriAI/face-antispoof-onnx - GitHub, дата последнего обращения: марта 12, 2026, <https://github.com/suriAI/face-antispoof-onnx/pulls>
31. What is the most reliable face liveness detection model and dataset for real-world mobile apps? - Stack Overflow, дата последнего обращения: марта 12, 2026,
<https://stackoverflow.com/questions/79861393/what-is-the-most-reliable-face-liveness-detection-model-and-dataset-for-real-wor>
32. Creation of liveness detection : r/computervision - Reddit, дата последнего обращения: марта 12, 2026,
https://www.reddit.com/r/computervision/comments/1mhlo0c/creation_of_liveness_detection/
33. Suri - GitHub, дата последнего обращения: марта 12, 2026,
<https://github.com/suriAI>